18 июня 2026 г. 20 реакций

Анонс TypeScript 7.0 RC

Дэниел Розенвассер
Главный менеджер по продуктам

Оглавление

Сегодня мы рады представить релиз-кандидат TypeScript 7.0!

Если вы не следили за разработкой TypeScript 7.0, то знайте, что этот релиз важен тем, что он построен на совершенно новой основе. В течение последнего года мы переносили существующую кодовую базу TypeScript с TypeScript (как самоподдерживающейся кодовой базы, компилируемой в JavaScript) на Go. Благодаря сочетанию скорости нативного кода и параллелизма с общей памятью **TypeScript 7.0 зачастую работает примерно в 10 раз быстрее**, чем TypeScript 6.0.

Чтобы получить новый компилятор, вы можете просто установить его из `typescript` пакета на npm, как и любой другой релиз:

Копировать

```
npm install -D typescript@rc
```

Новая кодовая база Go была методично перенесена из нашей существующей реализации, а не переписана с нуля, и ее логика проверки типов структурно идентична TypeScript 6.0. Такое архитектурное соответствие гарантирует, что компилятор будет обеспечивать ту же семантику, на которую вы уже полагаетесь. TypeScript 7.0 прошел проверку огромным набором тестов, который мы создали за десять лет, и уже используется в нескольких кодовых базах с многомиллионным кодом как внутри

Microsoft, так и за ее пределами. Он очень стабилен, хорошо совместим и готов к использованию в ваших повседневных рабочих процессах и конвейерах непрерывной интеграции *уже сегодня*.

Уже больше года мы работаем со многими внутренними командами Microsoft, а также с командами таких компаний, как Bloomberg, Canva, Figma, Google, Lattice, Linear, Miro, Notion, Slack, Vanta, Vercel, VoidZero и других, чтобы протестировать предварительные сборки TypeScript 7.0 на их кодовых базах. Отзывы в подавляющем большинстве положительные: многие команды сообщают о схожем ускорении работы, сокращении времени сборки на большую часть и более легком и плавном редактировании. В свою очередь, мы уверены, что релиз-кандидат в отличном состоянии, и нам не терпится, чтобы вы его опробовали.

Использование TypeScript 7.0 RC

Как уже упоминалось выше, чтобы получить TypeScript 7.0 RC, вы можете установить его через npm:

Копировать

```
npm install -D typescript@rc
```

После этого вы можете запустить `tsc` так же, как и с любой предыдущей версией TypeScript, и увидите те же результаты, что и раньше, только гораздо быстрее!

Копировать

```
> npx tsc --version
Version 7.0.1-rc
```

Чтобы опробовать возможности редактирования, вы можете установить расширение TypeScript Native Preview для VS Code. Поддержка редактора работает безупречно и уже несколько месяцев широко используется многими командами. Это простой и удобный способ сразу же опробовать TypeScript 7.0 в своей кодовой базе. Он основан на тех же принципах, что и работа с командной строкой, поэтому в редакторе вы получите те же преимущества в производительности, что и в командной строке. Примечательно, что он также основан на протоколе Language Server Protocol (LSP), что позволяет легко запускать его в большинстве современных редакторов и даже в таких инструментах, как Copilot CLI.

Совместная работа с TypeScript 6.0

Even though 7.0 RC is close to production-ready, we won't have a stable programmatic API available until at least several months from now with TypeScript 7.1. Given this, we have made it a priority to ensure TypeScript can be run side-by-side with TypeScript 6.0 for the foreseeable future without any conflicts around "which `tsc` is which?"

As part of the 6.0/7.0 transition process, we've published a new compatibility package, `@typescript/typescript6`. This package provides an executable named `tsc6`, so that if needed, you can install TypeScript 7.0 (which ships its own `tsc` binary) side-by-side without naming conflicts. The new package also re-exports the TypeScript 6.0 API, so that you can use `tsc` for TypeScript 7, while other tooling can continue to rely on 6.0.

Because some tools like typescript-eslint expect to import from `typescript` directly via peer dependencies, we recommend achieving this via npm aliases. You should be able to run the following command

Copy

```
npm install -D typescript@npm:@typescript/typescript6
```

or modify your `package.json` as follows:

Copy

```
{
  "devDependencies": {
    "typescript": "npm:@typescript/typescript6@^6.0.0",
  }
}
```

Note that doing this will leave you only with a `tsc6` executable. To get 7.0's `tsc`, you can add another alias for TypeScript 7 and `npx tsc` will just work with 7.0:

Copy

```
{
  "devDependencies": {
    "typescript": "npm:@typescript/typescript6@^6.0.0",
    "typescript-7": "npm:typescript@rc",
  }
}
```

Nightly Releases

TypeScript 7 nightlies are currently still being published under the `@typescript/native-preview` package on npm, and can be installed via

Copy

```
npm install -D @typescript/native-preview
```

The binary provided by that package is still named `tsgo`. Once TypeScript 7 is published with the `latest` tag on npm, we expect all stable releases, major pre-releases, and nightlies to be published under the `typescript` package on npm.

Parallelization and Controls

TypeScript 7.0 now performs many steps in parallel, including parsing, type-checking, and emitting. Some of these steps, like parsing and emitting can mostly be done independently across files. As such, parallelization automatically scales well with larger codebases with relatively little overhead. But not every step in a TypeScript build is easily parallelizable.

Checker Parallelization

Other steps, like type-checking, have more complex dependencies across files. Most files end up relying on the same type information from their dependencies and the global scope, and so running type-checkers completely independently would be wasteful – both in computation and memory. On the other hand, type-checking occasionally relies on the relative ordering of information in a program, and so type-checking from scratch must always check the same files in an identical order to ensure the same results.

To enable parallelization while avoiding these pitfalls, TypeScript 7.0 creates a fixed number of type-checker workers with their own view of the world. These type-checking workers may end up duplicating some common work, but given the same input files, they will always divide them identically and produce the same results.

The default number of type-checking workers is 4, but it can be configured with the new `--checkers` flag. You may find that increasing this number can further speed up builds on larger codebases where typical machines have more CPU cores, but will typically come at the cost of increased memory usage. Likewise, machines with fewer CPU cores and less memory (e.g. CI runners) may want to decrease this number to avoid unnecessary or incidental overhead.

In rare cases, varying the number of `--checkers` may surface order-dependent results. Specifying a fixed number of checkers across build environments can help ensure everyone is getting the same results, but is up to the discretion of each team.

Project Reference Builder Parallelization

TypeScript 7.0 can parallelize builds within a project, but it can now also build multiple projects at once as well. This behavior can be configured with the new `--builders` flag, which controls the number of parallel project reference builders that can run at once. This can be particularly helpful for monorepos with many projects.

Like `--checkers`, increasing the number of builders can speed up builds, but may come at the cost of increased memory usage. It also has a multiplicative effect with `--checkers`, so it's important to find the right balance for your machine and codebase. For example, building with `--checkers 4 --builders 4` allows up to 16 type-checkers to run at once, which may be excessive.

Unlike `--checkers`, varying the number of builders should not produce different results; however, building project references is fundamentally bottlenecked by the dependency graph of projects (with the exception of type-checking on codebases that leverage `--isolatedDeclarations` and separate syntactic declaration file emit).

Single-Threaded Mode

In some cases, it can be helpful to enforce single-threaded operation throughout the compiler. This may be useful for debugging, comparing performance with TypeScript 6 and 7, when orchestrating parallel builds externally, or for running in environments with very limited resources. To enable single-threaded mode, you can use the new `--singleThreaded` flag. This will not only cap the number of type-checking workers to 1, but also ensure parsing and emitting are done in a single thread.

Improved `--watch` Mode

Worth calling out is TypeScript 7's rebuilt `--watch` mode. `--watch` is now built on a new foundation derived from the Parcel bundler's file-watcher that provides efficient and stable cross-platform file watching capabilities.

When our team set out to port our file watching logic, we encountered a few challenges with cross-platform file watching in Go. The standard library doesn't provide a built-in file watching API, and existing third-party libraries we explored had various issues with stability, performance, cross-platform support, or issues with build tooling integration. We were able to build solutions around polling periodically to check for file changes, and this worked broadly across operating systems; however it was computationally expensive, especially at larger-scale

projects with many dependencies in `node_modules`. Even with dynamic scheduling strategies, we found that pure-polling solutions were too taxing for general use.

For many years, Visual Studio Code has relied on `@parcel/watcher`, and in recent years TypeScript in VS Code has relied on its file watching capabilities indirectly. While it seemed promising, one of the problems for us with Parcel's watcher is that it's written in C++, and in turn requires a full C++ toolchain to build. Given our positive experience with Parcel's watcher in VS Code, we explored porting it to Go with a few minimal assembly shims to avoid introducing a new toolchain dependency.

The exploration has been a success – what started as a very direct translation from C++ to Go was further refined into idiomatic Go that still passes the ported test suite. The watcher is a self-contained package that has allowed us to keep a clean separation of concerns between what we care to watch and why. We are now seeing significant resource improvements in `--watch` mode across platforms, and have been hearing positive feedback from earlier users of TypeScript 7.

We'd like to extend our thanks to Devon Govett whose work on Parcel has provided immense benefits to both the Visual Studio Code and TypeScript projects. We hope this port will provide opportunities and insights for the original Parcel watcher codebase over time.

Updates Since 5.x, and New Behaviors from 6.0

TypeScript 7.0 is made to be compatible with TypeScript 6.0's type-checking and command-line behavior. Practically any TypeScript code that compiles cleanly with TypeScript 6.0 (with the `stableTypeOrdering` flag on, and without any `ignoreDeprecations` flag set) should compile identically in TypeScript 7.0.

With that said, TypeScript 7.0 adopts 6.0's new defaults, and provides hard errors in the face of any flags and constructs deprecated in TypeScript 6.0. This is notable as 6.0 is still relatively new, and many projects will need to adapt to its new behaviors. We encourage developers to adopt TypeScript 6.0 to make the transition to TypeScript 7.0 easier, and you can also read the TypeScript 6.0 release blog post for more details on these deprecations.

At a glance, the notable default changes to configuration are:

- `strict` is `true` by default.
- `module` defaults to `esnext`.
- `target` defaults to the current stable ECMAScript version immediately preceding `esnext`.
- `noUncheckedSideEffectImports` is `true` by default.

- `libReplacement` is `false` by default.
- `stableTypeOrdering` is `true` by default, and cannot be turned off.
- `rootDir` now defaults to `./`, and inner source directories must be explicitly set.
- `types` now defaults to `[]`, and the old behavior can be restored by setting it to `["*"]`.

We believe the `rootDir` and `types` changes may be the most "surprising" changes, but they can be mitigated easily. Projects where the `tsconfig.json` sits outside of a directory like `src` will simply need to include `rootDir` to preserve the same directory structure.

Copy

```
  {
      "compilerOptions": {
          // ...
+         "rootDir": "./src"
      },
      "include": ["./src"]
  }
```

For the `types` change, projects that depend on specific global declarations will need to list them explicitly. For example,

Copy

```
  {
      "compilerOptions": {
          // Explicitly list the @types packages you need (e.g. bun,
+         "types": ["node", "jest"]
      }
  }
```

The deprecations that have turned into hard errors with no-op behavior are:

- `target: es5` is no longer supported.
- `downlevelIteration` is no longer supported.
- `moduleResolution: node/node10` are no longer supported, with `nodenext` and `bundler` being recommended instead.
- `module: amd, umd, systemjs, none` are no longer supported, with `esnext` or `preserve` being recommended in conjunction with bundlers or browser-based module resolution.

- `baseUrl` is no longer supported, and `paths` can be updated to be relative to the project root instead of `baseUrl`.
- `moduleResolution: classic` is no longer supported, and `bundler` or `nodenext` are the recommended replacements.
- `esModuleInterop` and `allowSyntheticDefaultImports` cannot be set to `false`.
- `alwaysStrict` is assumed to be `true` and can no longer be set to `false`.
- The `module` keyword cannot be used in namespace declarations.
- The `asserts` keyword cannot be used on imports, and must use the `with` keyword instead (to align with developments on ECMAScript's import attribute syntax).
- `/// <reference no-default-lib />` directives are no longer respected under `skipDefaultLibCheck`.
- Command line builds cannot take file paths when the current directory contains a `tsconfig.json` file unless passed an explicit `--ignoreConfig` flag.

Template Literal Types Now Preserve Unicode Code Points

TypeScript 7.0 now treats Unicode code points more naturally when inferring from template literal types. For example:

Copy

```
type HeadTail<S> = S extends `${infer Head}${infer Tail}` ? [Head, Ta

type Result = HeadTail<"😀abc">;
//   ^
// In 7.0: ["😀", "abc"]
// Previously: ["\ud83d", "\ude00abc"]
```

Previously, TypeScript followed JavaScript's UTF-16 indexing behavior here and split `"😀"` into two halves of a surrogate pair (`\ud83d` and `\ude00`). That was technically consistent with indexing in JavaScript (e.g. the inferred `Head` type was equal to `"😀abc"[0]`), but it usually wasn't what people intended, and could produce string literal types containing unpaired surrogates that aren't semantically meaningful.

This is a breaking change for type-level string manipulation that intentionally modeled UTF-16 code units, such as some string `Length` utilities. In practice, we expect the new behavior to be more useful and less surprising: template literal inference now follows the same intuition as iterating a string with `for...of` or spreading it with `[...str]`, where `"😀"` is treated as one unit.

JavaScript Differences

As we ported the existing codebase, we also took the opportunity to revisit how our JavaScript support works.

TypeScript originally supported JavaScript files by using JSDoc comments and recognizing certain code patterns for analysis and type inference. Lots of the time, this was based on popular coding patterns, but occasionally it was based on whatever people *might* be writing that Closure and the JSDoc doc generating tool might understand. While this approach was helpful for developers with loosely-written JSDoc codebases, it required a number of compromises and special cases to work well, and diverged in a number of ways from TypeScript's analysis in `.ts` files.

In TypeScript 7.0, we have reworked our JavaScript support to be more consistent with how we analyze TypeScript files. Some of the differences include:

- Values cannot be used where types are expected – instead, write `typeof someValue`
- `@enum` is not specially recognized anymore – create a `@typedef` on `(typeof YourEnumDeclaration)[keyof typeof YourEnumDeclaration]`.
- A standalone `?` is no longer usable as a type – use `any` instead.
- `@class` does not make a function a constructor – use a `class` declaration instead.
- Postfix `!` is not supported – just use `T`.
- Type names must be defined within a `@typedef` tag (i.e. `/** @typedef {T} TypeAliasName */`), not adjacent to an identifier (i.e. `/** @typedef {T} */ TypeAliasName;`).
- Closure-style function syntax (e.g. `function(string): void`) is no longer supported – use TypeScript shorthands instead (e.g. `(s: string) => void`).

Additionally, some JavaScript patterns, like aliasing `this` and reassigning the entirety of a function's `prototype` are no longer specially treated.

While some of our JS support is in flux, we have been updating this CHANGES.md file to capture the differences between TypeScript 6.0 and 7.0 in more detail.

Editor Experience

TypeScript 7.0's performance improvements are not limited to the command line experience – they also extend to the editor experience too. For VS Code users, the TypeScript Native Preview extension provides a seamless way to try out TypeScript 7.0 in your editor, and has seen widespread use. For Visual Studio users, the latest version of the editor will automatically enable TypeScript 7 based on your workspace. Of course, TypeScript 7 should work great in any

editor of your choosing. The new foundation is built on the Language Server Protocol (LSP) and is able to leverage multiple threads to serve simultaneous requests as quickly as possible.

Since it first debuted, we've added in missing functionality like auto-imports, expandable hovers, inlay hints, code lenses, go-to-source-definition, JSX linked editing and tag completions, and more. Missing features from TypeScript 7.0 beta, such as semantic highlighting, "sort imports", "remove unused imports", and more are now in.

Additionally, we've continued to drive performance and stability in the past few months. We've rebuilt much of our testing and diagnostics infrastructure to make sure the quality bar is high, in which we are able to fuzz-test the language server against the top TypeScript and JavaScript codebases on GitHub. Based on our data insights, we believe TypeScript 7 actually has reduced failing language server commands by over 20x compared to TypeScript 6.0.

This extension respects most of the same configuration settings as the built-in TypeScript extension for Visual Studio Code, along with most of the same features.

The Road to TypeScript 7.0

With TypeScript 7.0 RC now available, our current plan is to release TypeScript 7.0 within the next month, and we will be focusing on release coordination and logistics, reported regressions, and future API capabilities in TypeScript 7.1.

Between now and then, we would especially appreciate feedback from trying TypeScript 7.0 on real projects. If you run into any issues, please let us know on the issue tracker for microsoft/typescript-go so we can make sure the stable release is in great shape.

We also encourage you to share your experience using TypeScript 7.0 and tag @typescriptlang.org on Bluesky or @typescript@fosstodon.org on Mastodon, or @typescript on Twitter.

Our team is incredibly excited for you to try this release out, so try it today and let us know what you think. Happy hacking!

– The TypeScript Team

Category

TypeScript

Share

Author

Daniel Rosenwasser
Principal Product Manager

Daniel Rosenwasser is the product manager of the TypeScript team. He has a passion for programming languages, compilers, and great developer tooling.

4 comments

Join the discussion.

Sign in

Sort by : Newest

Umesh Neupane June 28, 2026 0

Why typescript never enforces static fields initialization unlike instance fields?

```
class MathUtil {
static readonly PI: number; // typescript doesn't care whether I initialize this field or not
}

let r = 7;
let area = MathUtil.PI * r * r; // typescript doesn't give a damn here about type safety
```

Log in to Vote or Reply

Vasyl Zuziak 7 days ago 0

is it possible to add some automation that would (re)publish all 6.x.x version to '@typescript/typescript6' package? because for it latest version is 6.0.1 and for regular typescript we already have 6.0.3?

Log in to Vote or Reply

stav alfi June 25, 2026 0

What about massive ram usage of the watcher?

Log in to Vote or Reply

David De Sloovere June 19, 2026 1

Together with oxlint and oxfmt, this typescript rewrite will speed up development greatly. It's awesome to see companies and individuals invest in this to help a whole industry forward.
Thank you Microsoft and VoidZero! ❤️

Log in to Vote or Reply

Load more comments

Read next

April 21, 2026

Announcing TypeScript 7.0 Beta

Daniel Rosenwasser

March 23, 2026

Announcing TypeScript 6.0

Daniel Rosenwasser

Stay informed

Get notified when new posts are published.

Email *

Enter your email

Country/Region *

Select...

I would like to receive the TypeScript Newsletter. Privacy Statement.

Subscribe

Follow this blog